



Università
di **Genova**

DIBRIS: Robotics Engineering

AIRO2 PRESENTATION

ASSIGNMENT D5-V2: AGRICULTURAL ROBOTICS - IRRIGATION PLANNING

Student

Sisani Francesco

Teacher

Mastrogiovanni Fulvio

A.A. 25/26

PROBLEM DEFINITION AND OBJECTIVES

Problem: one robot is deployed in a field divided in crops, as shown in figure 1.

It must be able to bring all the crops to a certain moisture level, assumed as stable, while keeping count of environmental constraints and limited resources

Assumptions:

- robot always starts in start₁;
- locations connected in the four directions;
- moisture level represented in numbers, 50 is the stability threshold

Objective: define solution plans using features from basic PDDL and PDDL+ to:

- use the planner's capabilities;
- explain the course of action that brings to an approximation of realistic scenarios.

Planner used: ENSHP (online and local)

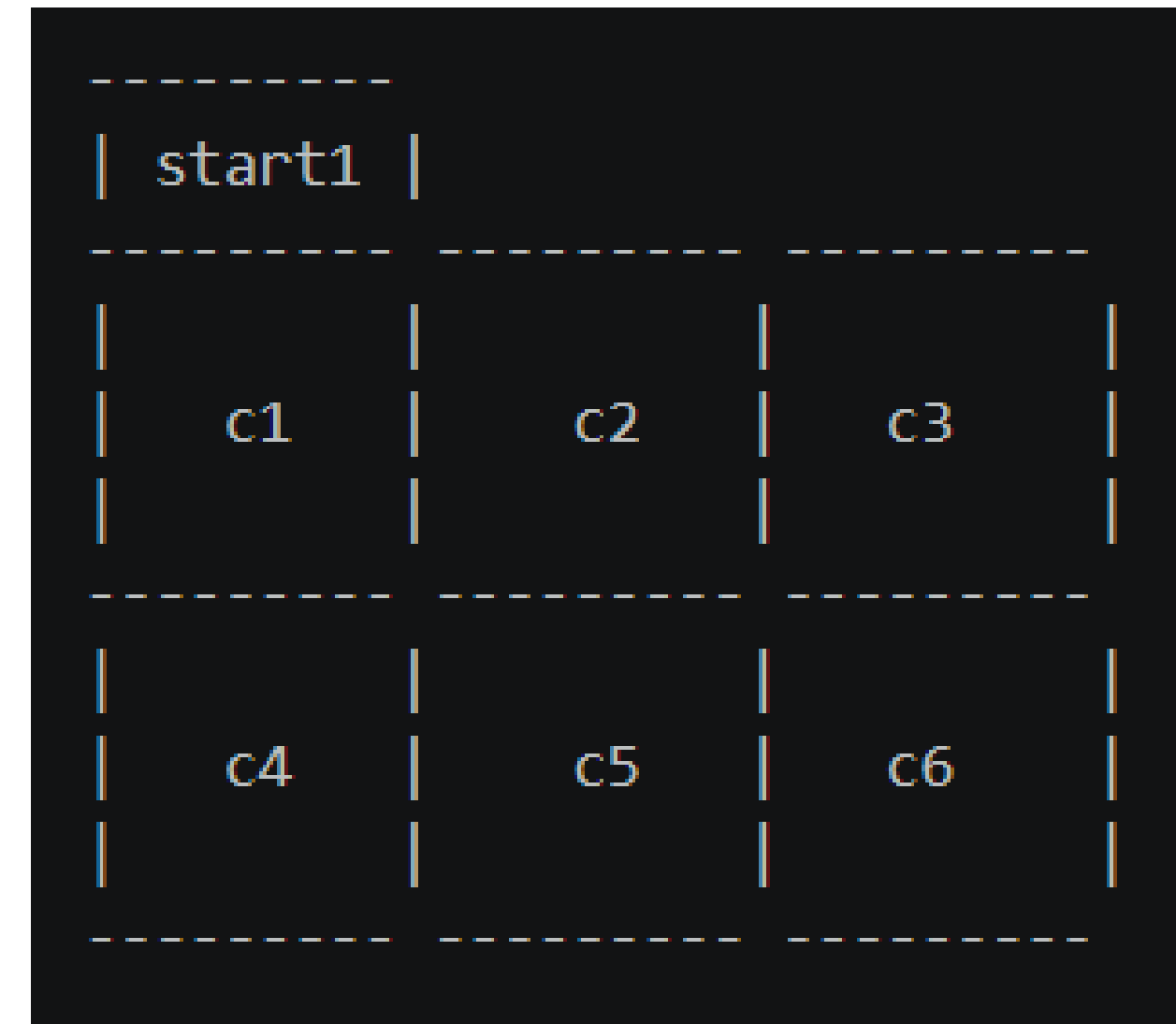


Fig 1: Testing environment

BASIC PDDL DOMAIN

Domain file common to both the abundant and limited resource problem files.

No environmental dynamics involved, used to define the robot's reasoning logic.

Main actions:

1) *Movement*: used to change the robot's position in the field. Allowed between two connected crops.

2) *Check_levels*: checks every crop's moisture level and chooses the one with the lowest level.

3) *Irrigate*: when the robot is located on a prioritized crop, it irrigates that crop by decreasing its water supply by 10 and increasing that crop's moisture level by 10. Numeric fluents avoid binary representation.

4) *Sacrifice*: a crop is flagged as sacrificed when the robot cannot irrigate it due to complete resource depletion.

```
(:action check_levels
  :parameters (?c - crop)
  :precondition (and
    (forall (?other - crop) (>= (moisture_level ?other)
      (moisture_level ?c)))
  )
  :effect (and
    (is-priority ?c)
    (forall (?other - crop)
      (when (not (= ?other ?c)) (not (is-priority ?other)))
    )
  )
)
```

Fig. 2: code structure of priority assignment

Robot logic: Choose priority crop → Reach it → Irrigate it → Repeat

PDDL+ DOMAIN

PDDL+ allows, by introducing continuous time, to add more realism in the current scenario.

Two main modifications emerge when compared to the basic scenario:

- introduced a process which passively drains every crop's moisture level and an event which makes a crop unusable if left alone for too much time;
- modified the logic that regulates movement and irrigation: they are no more instant actions, but processes toggled by actions and events;
- modified the priority logic to make it less computationally heavy in continuous time.

```
(:process evaporation
  :parameters (?c - crop)
  :precondition (and (>= (moisture_level ?c) 10) (not (unusable ?c)))
  :effect (and
    (decrease (moisture_level ?c) (* 1 #t))
  )
)

(:event drought
  :parameters (?c - crop)
  :precondition (and
    (<= (moisture_level ?c) 10)
    (not (unusable ?c))
  )
  :effect (and
    (assign (moisture_level ?c) 0)
    (unusable ?c)
    (increase (num_drought_events) 1)
  )
)
```

Fig.5: environmental conditions implemented

RESULTS - ABUNDANT WATER PROBLEM

PDDL Editor

File Session

planning.domains

Irrigation_basic_domain.pddl

Irrigation_limited_problem.pddl

Irrigation_problem.pddl

```
1 (define (problem irr
2
3 (:domain irrigation_
4
5 (:objects robot1 - r
6       c1 c2 c3 c
7       start1 - d
8 )
9
10 (:init
11
12 (= (water_supply
13 (= (num_sacrific
14 (needs_check)
15
16 (at robot1 start
17 (connected start
18 (connected c1 st
19 (connected c1 c2
20 (connected c2 c1
21 (connected c1 c4
22 (connected c4 c1
23 (connected c2 c3
24 (connected c3 c2
25 (connected c2 c5
26 (connected c5 c2
27 (connected c6 c3
28 (connected c3 c6
29 (connected c6 c5
30 (connected c5 c6)
31 (connected c4 c5)
32 (connected c5 c4)
33
34 (= (moisture_level c1) 17)
35 (= (moisture_level c2) 63)
36 (= (moisture_level c3) 42)
37 (= (moisture_level c4) 88)
38 (= (moisture_level c5) 5)
39 (= (moisture_level c6) 71)
40
41 )
```

Compute plan

Domain: Irrigation_basic_domain.pddl

Problem: Irrigation_limited_problem.pddl

Solver: ENHSP: Expressive Numeric H

Description: Supports classical and numeric planning (PDDL2.1), optimal simple numeric planning, satisficing non-linear numeric planning, autonomous processes and discrete events (PDDL+), and more.

Plan Cancel

RESULTS - LIMITED WATER PROBLEM

PDDL Editor

File Session

planning.domains

Irrigation_basic_domain.pddl

Irrigation_limited_problem.pddl

Irrigation_problem.pddl

```
1 (define (problem irr
2
3 (:domain irrigation_
4
5 (:objects robot1 - r
6       c1 c2 c3 c4
7       start1 - de
8 )
9
10 (:init
11
12   (= (water_supply
13     (= (num_sacrific
14       (needs_check)
15
16   (at robot1 start
17   (connected start
18   (connected c1 st
19   (connected c1 c2
20   (connected c2 c1
21   (connected c1 c4
22   (connected c4 c1
23   (connected c2 c3
24   (connected c3 c2
25   (connected c2 c5
26   (connected c5 c2
27   (connected c6 c3
28   (connected c3 c6
29   (connected c6 c5
30   (connected c5 c6)
31   (connected c4 c5)
32   (connected c5 c4)
33
34   (= (moisture_level c1) 17)
35   (= (moisture_level c2) 63)
36   (= (moisture_level c3) 42)
37   (= (moisture_level c4) 88)
38   (= (moisture_level c5) 5)
39   (= (moisture_level c6) 71)
40
41 )
```

Compute plan

Domain: Irrigation_basic_domain.pddl

Problem: Irrigation_limited_problem.pddl

Solver: ENHSP: Expressive Numeric H

Description: Supports classical and numeric planning (PDDL2.1), optimal simple numeric planning, satisficing non-linear numeric planning, autonomous processes and discrete events (PDDL+), and more.

Plan Cancel

RESULTS - PDDL+

PDDL Editor File Session planning.domains

Irrigation_basic_domain.pddl

Irrigation_limited_problem.pddl

Irrigation_problem.pddl

Irrigation_domain_plus.pddl

Irrigation_problem_plus.pddl

```
1 (define (problem irr
2
3 (:domain irrigation_
4
5 (:objects robot - ro
6       c1 c2 c3 c4
7       start1 - d
8 )
9
10 (:init
11
12 (= (water_supply
13 (= (move_progres
14 (= (num_drought_
15
16 (at robot start1
17 (idle robot)
18
19 (connected start
20 (connected c1 st
21 (connected c1 c2
22 (connected c2 c1
23 (connected c1 c4
24 (connected c4 c1
25 (connected c2 c3
26 (connected c3 c2
27 (connected c2 c5
28 (connected c5 c2
29 (connected c6 c3
30 (connected c3 c6)
31 (connected c6 c5)
32 (connected c5 c6)
33 (connected c4 c5)
34 (connected c5 c4)
35
36 (= (moisture_level c1) 27)
37 (= (moisture_level c2) 63)
38 (= (moisture_level c3) 42)
39 (= (moisture_level c4) 88)
40 (= (moisture_level c5) 25)
41 (= (moisture_level c6) 71)
```

Compute plan

Domain Irrigation_basic_domain.pddl ▾

Problem Irrigation_limited_problem.pc ▾

Solver ENHSP: Expressive Numeric H ▾

Description Supports classical and numeric planning (PDDL2.1), optimal simple numeric planning, satisficing non-linear numeric planning, autonomous processes and discrete events (PDDL+), and more.

Plan **Cancel**

CONCLUSIONS

Main challenges:

- Implementing the priority logic and solving related bottlenecks in both scenarios
- Conversion of movement and irrigation actions in continuous time
- Ordering the actions in both scenarios to make the course of action logically sound

Possible differences from current implementation:

- Priority logic in basic problems is fired after every irrigation step, rather than after a complete irrigation.
Possible to modify it since no environmental dynamics are involved.
- In PDDL+, movement is regulated by a action → process → event logic, while irrigation has a action → process → action logic.

Limitations:

- All problems' solvability is hard-constrained by the water supply's quantity
- Known issue: first action in PDDL+ fires after two seconds